Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 4

з дисципліни: «Теорія алгоритмів»

Тема: «Робота з хеш-таблицями»

Варіант № 2

Виконав:
Студент групи АІ-254
Гармаш Г.Є.

Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: Метою виконання лабораторної роботи є набуття практичних навичок із проєктування, реалізації, тестування та аналізу алгоритмів створення і обробки хеш-таблиць: хешування з роздільними ланцюжками (відкрите хешування) та хешування з відкритою адресацією (закрите хешування) з хеш-функціями, побудованими методами ділення та множення.

Завдання:

1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм створення відкритої хеш-таблиці при хешуванні з ланцюжками за варіантом завдання, використовуючи хеш-функції за методом ділення $h(k) = k \bmod 13$ та множення $h(k) = \lfloor 16 \cdot (kA \bmod 1) \rfloor$.
2. Виконати візуалізацію створення відкритої хеш-таблиці, надати пояснення.
3. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм створення закритої хеш-таблиці при хешуванні з відкритою адресацією за варіантом завдання, використовуючи ті самі хеш-функції.
4. Виконати візуалізацію створення закритої хеш-таблиці, надати пояснення.
5. Знайти кількість порівнянь при пошуку (видаленні) елементів у побудованих хеш-таблицях, вказати елемент з максимальною кількістю порівнянь та підрахувати середню кількість порівнянь у кожній хеш-таблиці.

2	Не той урожай, що в полі, а той, що в коморі.

Вхідні дані та хеш-функції

Вхідне прислів'я містить 11 слів, серед яких «той», «що» та «в» повторюються по двічі. У словник кожен ключ потрапляє один раз, тому в хеш-таблицю заносяться 8 унікальних слів у порядку першої появи: не, той, урожай, що, в, полі, а, коморі. Ключем k слова є сума номерів позицій його літер в українському алфавіті (рис. 4.3 методички). Для методу ділення розмір таблиці $m = 13$, $h(k) = k \bmod 13$. Для методу множення $m = 16$, $h(k) = \lfloor 16 \cdot (kA \bmod 1) \rfloor$, де $A = 0{,}6180339887$ (константа Кнута). Значення хеш-функцій наведено в таблиці 4.1.

Таблиця 4.1 – Значення хеш-функцій для слів варіанту

№	Слово	Обчислення k	k	h(k) = k mod 13	kA mod 1	h(k) = ⌊16·(kA iod 1)⌋
1	не	н18 + е7 = 25	25	12	0,4508	⌊16·0,4508⌋ = 7
2	той	т23 + о19 + й14 = 56	56	4	0,6099	⌊16·0,6099⌋ = 9
3	урожай	у24 + р21 + о19 + ж9 + а1 + й14 = 88	88	10	0,3870	⌊16·0,3870⌋ = 6
4	що	щ30 + о19 = 49	49	10	0,2837	⌊16·0,2837⌋ = 4
5	в	в3 = 3	3	3	0,8541	⌊16·0,8541⌋ = 13
6	полі	п20 + о19 + л16 + і12 = 67	67	2	0,4083	⌊16·0,4083⌋ = 6

7	а	а1 = 1	1	1	0,6180	⌊ 16·0,6180⌋ = 9
8	коморі	к15 + о19 + м17 + о19 + р21 + і12 = 103	103	12	0,6575	⌊16·0,6575⌋ = 10

Колізії: для методу ділення h(урожай) = h(що) = 10 та h(не) = h(коморі) = 12; для методу множення h(урожай) = h(полі) = 6 та h(той) = h(а) = 9.

1. Хешування з роздільними ланцюжками (відкрите хешування)

Кожна комірка таблиці містить список (ланцюжок) ключів, що мають однакове хеш-значення. Новий ключ додається в кінець ланцюжка комірки h(k). Таблиця 4.2 сформована вручну за значеннями таблиці 4.1.

Таблиця 4.2 – Вигляд хеш-таблиці при хешуванні з ланцюжками

Індекс	Метод ділення (m = 13)	Метод множення (m = 16)
0		
1	а	
2	полі	
3	в	
4	той	що
5		
6		урожай → полі
7		не
8		
9		той → а
10	урожай → що	коморі
11		
12	не → коморі	
13	—	в
14	—	
15	—	

Лістинг 4.1 – Python-код реалізації алгоритму хешування з роздільними ланцюжками та виведення відкритої хеш-таблиці.

```
import math


M_DIV = 13
M_MUL = 16
A = 0.6180339887


TEXT = "Не той урожай, що в полі, а той, що в коморі."
```

```
LETTER_POSITIONS = {
    'а': 1, 'б': 2, 'в': 3, 'г': 4, 'ґ': 5, 'д': 6, 'е': 7, 'є': 8,
    'ж': 9, 'з': 10, 'и': 11, 'і': 12, 'ї': 13, 'й': 14, 'к': 15,
    'л': 16, 'м': 17, 'н': 18, 'о': 19, 'п': 20, 'р': 21, 'с': 22,
    'т': 23, 'у': 24, 'ф': 25, 'х': 26, 'ц': 27, 'ч': 28, 'ш': 29,
    'щ': 30, 'ь': 31, 'ю': 32, 'я': 33
}


def get_words(text):
    words = []
    for word in text.lower().replace(",", "").replace(".", "").split():
        if word not in words:
            words.append(word)
    return words


def key_value(word):
    total = 0
    for char in word:
        total += LETTER_POSITIONS.get(char, 0)
    return total


def hash_division(k):
    return k % M_DIV


def hash_multiplication(k):
    fraction = (k * A) % 1
    return math.floor(M_MUL * fraction)


def build_open_hash_table(words, m, hash_func):
    hash_table = [[] for _ in range(m)]
    for word in words:
        address = hash_func(key_value(word))
        hash_table[address].append(word)
    return hash_table


def display_hash_table(table):
    for i, chain in enumerate(table):
        print(f"Індекс {i:02d}: {chain}")


WORDS = get_words(TEXT)
print("Слова:", WORDS)

print("\n--- Значення хеш-функцій ---")
for word in WORDS:
    k = key_value(word)
    print(f"{word}: k = {k}, h_div = {hash_division(k)}, h_mul =
{hash_multiplication(k)}")

print("\n--- Метод ділення (M=13) ---")
table_div = build_open_hash_table(WORDS, M_DIV, hash_division)
display_hash_table(table_div)

print("\n--- Метод множення (M=16) ---")
table_mul = build_open_hash_table(WORDS, M_MUL, hash_multiplication)
display_hash_table(table_mul)
```

Таблиця 4.3 – Результати побудови відкритої хеш-таблиці

```
Слова: ['не', 'той', 'урожай', 'що', 'в', 'полі', 'а', 'коморі']
--- Значення хеш-функцій ---
не: k = 25, h_div = 12, h_mul = 7
той: k = 56, h_div = 4, h_mul = 9
урожай: k = 88, h_div = 10, h_mul = 6
що: k = 49, h_div = 10, h_mul = 4
в: k = 3, h_div = 3, h_mul = 13
полі: k = 67, h_div = 2, h_mul = 6
а: k = 1, h_div = 1, h_mul = 9
коморі: k = 103, h_div = 12, h_mul = 10
--- Метод ділення (M=13) ---
Індекс 00: []
Індекс 01: ['а']
Індекс 02: ['полі']
Індекс 03: ['в']
Індекс 04: ['той']
Індекс 05: []
Індекс 06: []
Індекс 07: []
Індекс 08: []
Індекс 09: []
Індекс 10: ['урожай', 'що']
Індекс 11: []
Індекс 12: ['не', 'коморі']
--- Метод множення (M=16) ---
Індекс 00: []
Індекс 01: []
Індекс 02: []
Індекс 03: []
Індекс 04: ['що']
Індекс 05: []
Індекс 06: ['урожай', 'полі']
Індекс 07: ['не']
Індекс 08: []
Індекс 09: ['той', 'а']
Індекс 10: ['коморі']
Індекс 11: []
Індекс 12: []
Індекс 13: ['в']
Індекс 14: []
Індекс 15: []
```

Результат побудови відкритої хеш-таблиці за допомогою Python-коду (Табл. 4.3) збігається з таблицею, побудованою вручну (Табл. 4.2).

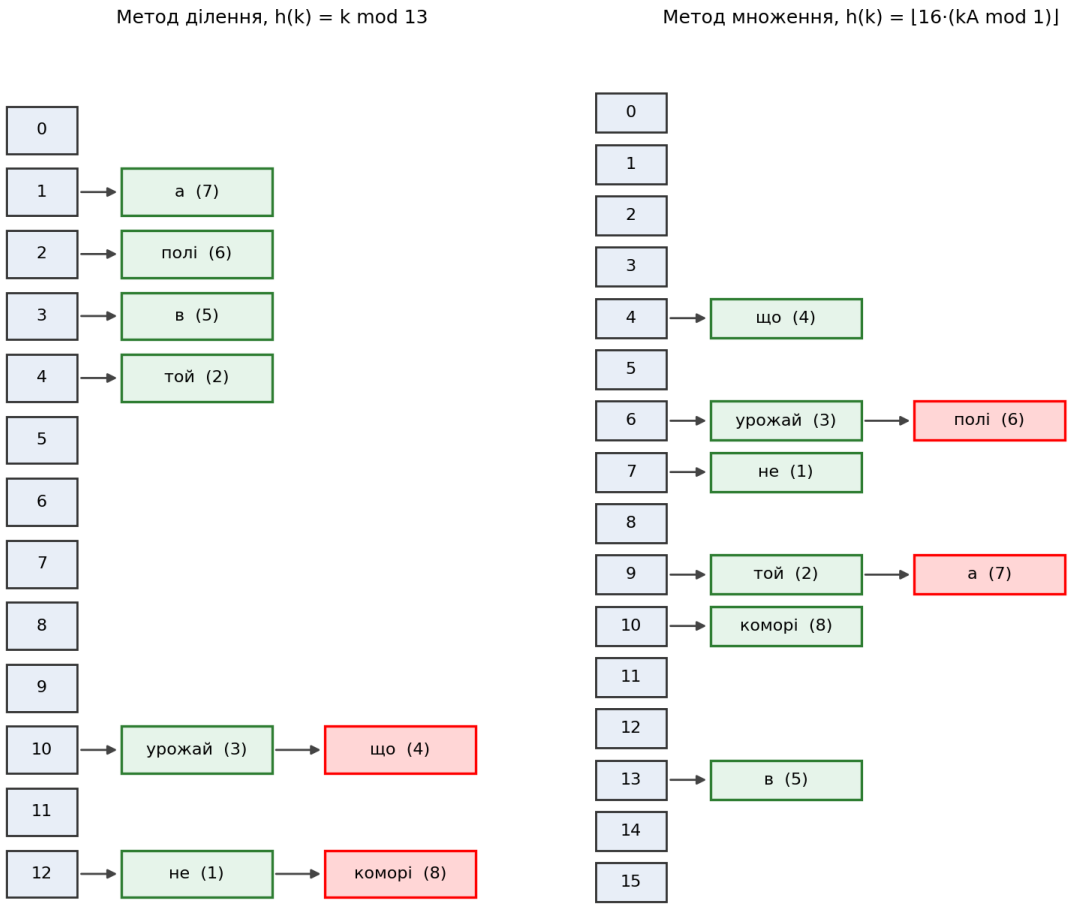


Рисунок 4.1 – Візуалізація відкритої хеш-таблиці (зелений – перший елемент ланцюжка, червоний – елемент, доданий після колізії; у дужках номер вставки).

На рисунку 4.1 показано створення відкритої хеш-таблиці двома хеш-функціями. Кожне слово вставляється в ланцюжок комірки h(k). У методі ділення колізії виникли в комірках 10 (урожай → що) та 12 (не → коморі), у методі множення – в комірках 6 (урожай → полі) та 9 (той → а). Довжина найдовшого ланцюжка в обох таблицях дорівнює 2, решта комірок містить не більше одного ключа. Коефіцієнт заповнення $\alpha = n/m$ дорівнює $8/13 \approx 0{,}62$ для методу ділення та $8/16 = 0{,}5$ для методу множення.

2. Хешування з відкритою адресацією (закрите хешування)

Усі ключі зберігаються безпосередньо в таблиці. У разі колізії застосовується лінійне дослідження h(k, i) = (h(k) + i) mod m, і таблиця розглядається як циклічний масив. Таблиці 4.4 і 4.5 сформовано вручну: у стовпці кожного кроку показано стан таблиці після вставки слова (зеленим позначено слово, що стало на місце h(k), червоним – слово, зміщене через колізію).

Таблиця 4.4 – Покрокова побудова хеш-таблиці з відкритою адресацією (метод ділення, m = 13)

Крок Слово h(k)	**1** не 12	**2** той 4	**3** урожай 10	**4** що 10	**5** в 3	**6** полі 2	**7** а 1	**8** коморі 12
0								**коморі**
1							**а**	а
2						**полі**	полі	полі
3					**в**	в	в	в
4		**той**	той	той	той	той	той	той
5								
6								
7								
8								
9								
10			**урожай**	урожай	урожай	урожай	урожай	урожай
11				**що**	що	що	що	що
12	**не**	не	не	не	не	не	не	не

Таблиця 4.5 – Покрокова побудова хеш-таблиці з відкритою адресацією (метод множення, m = 16)

Крок Слово h(k)	**1** не 7	**2** той 9	**3** урожай 6	**4** що 4	**5** в 13	**6** полі 6	**7** а 9	**8** коморі 10
0								
1								
2								
3								
4				**що**	що	що	що	що
5								
6			**урожай**	урожай	урожай	урожай	урожай	урожай
7	**не**	не	не	не	не	не	не	не
8						**полі**	полі	полі
9		**той**	той	той	той	той	той	той
10							**а**	а
11								**коморі**
12								

Крок Слово h(k)	**1** не 7	**2** той 9	**3** урожай 6	**4** що 4	**5** в 13	**6** полі 6	**7** а 9	**8** коморі 10
13					**в**	в	в	в
14								
15								

Лістинг 4.2 – Python-код реалізації алгоритму хешування з відкритою адресацією та виведення закритої хеш-таблиці.

```
import math

M_DIV = 13
M_MUL = 16
A = 0.6180339887

TEXT = "Не той урожай, що в полі, а той, що в коморі."

LETTER_POSITIONS = {
    'а': 1, 'б': 2, 'в': 3, 'г': 4, 'ґ': 5, 'д': 6, 'е': 7, 'є': 8,
    'ж': 9, 'з': 10, 'и': 11, 'і': 12, 'ї': 13, 'й': 14, 'к': 15,
    'л': 16, 'м': 17, 'н': 18, 'о': 19, 'п': 20, 'р': 21, 'с': 22,
    'т': 23, 'у': 24, 'ф': 25, 'х': 26, 'ц': 27, 'ч': 28, 'ш': 29,
    'щ': 30, 'ь': 31, 'ю': 32, 'я': 33
}


def get_words(text):
    words = []
    for word in text.lower().replace(",", "").replace(".", "").split():
        if word not in words:
            words.append(word)
    return words


def key_value(word):
    total = 0
    for char in word:
        total += LETTER_POSITIONS.get(char, 0)
    return total


def hash_division(k):
    return k % M_DIV


def hash_multiplication(k):
    fraction = (k * A) % 1
    return math.floor(M_MUL * fraction)


def build_closed_hash_table(words, m, hash_func):
    hash_table = [None] * m
    for word in words:
        start = hash_func(key_value(word))
        for i in range(m):
            address = (start + i) % m
            if hash_table[address] is None:
                hash_table[address] = word
```

```
                print(f"{word}: h = {start}, комірка {address}, досліджень:
{i + 1}")
                break
        else:
            print(f"Помилка: таблиця заповнена, слово {word} не додано")
    return hash_table


def display_hash_table(table):
    print("Індекс | Слово")
    print("-------|-------")
    for i, item in enumerate(table):
        value = item if item is not None else "(NULL)"
        print(f"{i:02d}     | {value}")


WORDS = get_words(TEXT)
print("Слова:", WORDS)

print("\n--- Метод ділення (M=13) ---")
table_div = build_closed_hash_table(WORDS, M_DIV, hash_division)
display_hash_table(table_div)

print("\n--- Метод множення (M=16) ---")
table_mul = build_closed_hash_table(WORDS, M_MUL, hash_multiplication)
display_hash_table(table_mul)
```

Таблиця 4.6 – Результати побудови хеш-таблиці з відкритою адресацією

```
Слова: ['не', 'той', 'урожай', 'що', 'в', 'полі', 'а', 'коморі']
--- Метод ділення (M=13) ---
не: h = 12, комірка 12, досліджень: 1
той: h = 4, комірка 4, досліджень: 1
урожай: h = 10, комірка 10, досліджень: 1
що: h = 10, комірка 11, досліджень: 2
в: h = 3, комірка 3, досліджень: 1
полі: h = 2, комірка 2, досліджень: 1
а: h = 1, комірка 1, досліджень: 1
коморі: h = 12, комірка 0, досліджень: 2
Індекс | Слово
-------|-------
00     | коморі
01     | а
02     | полі
03     | в
04     | той
05     | (NULL)
06     | (NULL)
07     | (NULL)
08     | (NULL)
09     | (NULL)
10     | урожай
11     | що
12     | не
--- Метод множення (M=16) ---
не: h = 7, комірка 7, досліджень: 1
той: h = 9, комірка 9, досліджень: 1
урожай: h = 6, комірка 6, досліджень: 1
що: h = 4, комірка 4, досліджень: 1
в: h = 13, комірка 13, досліджень: 1
полі: h = 6, комірка 8, досліджень: 3
а: h = 9, комірка 10, досліджень: 2
```

```
коморі: h = 10, комірка 11, досліджень: 2
Індекс | Слово
-------|-------
00     | (NULL)
01     | (NULL)
02     | (NULL)
03     | (NULL)
04     | що
05     | (NULL)
06     | урожай
07     | не
08     | полі
09     | той
10     | а
11     | коморі
12     | (NULL)
13     | в
14     | (NULL)
15     | (NULL)
```

Результат побудови закритої хеш-таблиці за допомогою Python-коду (Табл. 4.6) збігається з таблицями, побудованими вручну (Табл. 4.4 та 4.5).

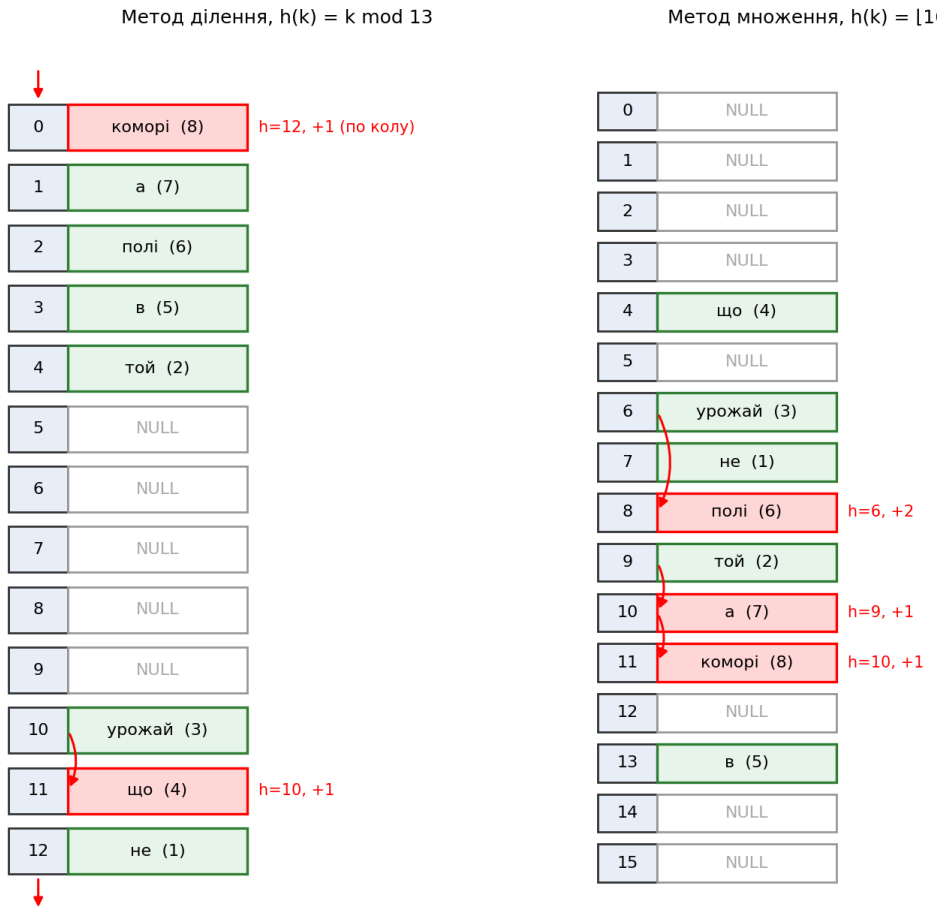


Рисунок 4.2 – Візуалізація закритої хеш-таблиці (червоним позначено слова, зміщені через колізію; h – початкова комірка, +n – кількість зсувів).

На рисунку 4.2 показано результат вставки слів із лінійним дослідженням. У методі ділення слово «що» (h = 10) знайшло комірку 10 зайнятою словом «урожай» і стало в комірку 11. Слово «коморі» (h = 12) знайшло комірку 12 зайнятою словом «не» і за циклічним правилом перейшло на початок таблиці, в комірку 0. У методі множення слово «полі» (h = 6) проминуло зайняті комірки 6 і 7 та стало в комірку 8, слово «а» (h = 9) – у комірку 10, а слово «коморі» (h = 10) через зайняту комірку 10 опинилося в комірці 11. Таким чином, у ланцюжку комірок 9–11 утворився кластер із трьох послідовних ключів, що характерно для лінійного дослідження.

3. Аналіз ефективності пошуку (видалення) в хеш-таблицях

Кількість порівнянь підраховувалась для кожного елемента побудованих таблиць. У таблиці з ланцюжками порівнянням вважається перевірка ключа в ланцюжку комірки h(k) до знаходження шуканого слова. У таблиці з відкритою адресацією порівнянням вважається кожна переглянута комірка, що містить ключ (дослідження починається з комірки h(k) і триває циклічно). Видалення вимагає тих самих порівнянь, що й пошук, бо спочатку потрібно

знайти елемент: у ланцюжку його потім вилучають зі списку, а при відкритій адресації комірку позначають спеціальним значенням DELETED.

Лістинг 4.3 – Python-код підрахунку кількості порівнянь при пошуку елементів у хеш-таблицях.

```
import math

M_DIV = 13
M_MUL = 16
A = 0.6180339887

TEXT = "Не той урожай, що в полі, а той, що в коморі."

LETTER_POSITIONS = {
    'а': 1, 'б': 2, 'в': 3, 'г': 4, 'ґ': 5, 'д': 6, 'е': 7, 'є': 8,
    'ж': 9, 'з': 10, 'и': 11, 'і': 12, 'ї': 13, 'й': 14, 'к': 15,
    'л': 16, 'м': 17, 'н': 18, 'о': 19, 'п': 20, 'р': 21, 'с': 22,
    'т': 23, 'у': 24, 'ф': 25, 'х': 26, 'ц': 27, 'ч': 28, 'ш': 29,
    'щ': 30, 'ь': 31, 'ю': 32, 'я': 33
}


def get_words(text):
    words = []
    for word in text.lower().replace(",", "").replace(".", "").split():
        if word not in words:
            words.append(word)
    return words


def key_value(word):
    total = 0
    for char in word:
        total += LETTER_POSITIONS.get(char, 0)
    return total


def hash_division(k):
    return k % M_DIV


def hash_multiplication(k):
    fraction = (k * A) % 1
    return math.floor(M_MUL * fraction)


def build_open_hash_table(words, m, hash_func):
    hash_table = [[] for _ in range(m)]
    for word in words:
        hash_table[hash_func(key_value(word))].append(word)
    return hash_table


def build_closed_hash_table(words, m, hash_func):
    hash_table = [None] * m
    for word in words:
        start = hash_func(key_value(word))
        for i in range(m):
            address = (start + i) % m
            if hash_table[address] is None:
                hash_table[address] = word
```

```
                break
    return hash_table


def search_open(table, word, hash_func):
    comparisons = 0
    for item in table[hash_func(key_value(word))]:
        comparisons += 1
        if item == word:
            break
    return comparisons


def search_closed(table, word, hash_func):
    m = len(table)
    start = hash_func(key_value(word))
    comparisons = 0
    for i in range(m):
        item = table[(start + i) % m]
        if item is None:
            break
        comparisons += 1
        if item == word:
            break
    return comparisons


def report(title, table, words, hash_func, search_func):
    print(f"\n--- {title} ---")
    total = 0
    max_comparisons = 0
    max_word = ""
    for word in words:
        c = search_func(table, word, hash_func)
        print(f"{word}: {c}")
        total += c
        if c > max_comparisons:
            max_comparisons = c
            max_word = word
    print(f"Максимум порівнянь: {max_comparisons} (елемент '{max_word}')")
    print(f"Сума порівнянь: {total}")
    print(f"Середня кількість порівнянь: {total / len(words):.2f}")


WORDS = get_words(TEXT)

open_div = build_open_hash_table(WORDS, M_DIV, hash_division)
open_mul = build_open_hash_table(WORDS, M_MUL, hash_multiplication)
closed_div = build_closed_hash_table(WORDS, M_DIV, hash_division)
closed_mul = build_closed_hash_table(WORDS, M_MUL, hash_multiplication)

report("Ланцюжки, метод ділення (M=13)", open_div, WORDS, hash_division,
search_open)
report("Ланцюжки, метод множення (M=16)", open_mul, WORDS,
hash_multiplication, search_open)
report("Відкрита адресація, метод ділення (M=13)", closed_div, WORDS,
hash_division, search_closed)
report("Відкрита адресація, метод множення (M=16)", closed_mul, WORDS,
hash_multiplication, search_closed)
```

Таблиця 4.7 – Кількість порівнянь при пошуку (видаленні) елементів

Слово	Ланцюжки, ділення (m = 13)	Ланцюжки, множення (m = 16)	Відкрита адресація, ділення (m = 13)	Відкрита адресація, множення (m = 16)
не	1	1	1	1
той	1	1	1	1
урожай	1	1	1	1
що	2	1	2	1
в	1	1	1	1
полі	1	2	1	3
а	1	2	1	2
коморі	2	1	2	2
Сума порівнянь	10	10	10	12
Максимум (елемент)	2 (що, коморі)	2 (полі, а)	2 (що, коморі)	3 (полі)
Середня кількість	1,25	1,25	1,25	1,50

Таблиця 4.8 – Порівняння ефективності хешування з ланцюжками та з відкритою адресацією

Метод	α = n/m	Середнє, експеримент	Середнє, теорія	Максимум	Найгірший випадок
Ланцюжки, ділення (m = 13)	0,62	1,25	1,31	2	O(n)
Ланцюжки, множення (m = 16)	0,50	1,25	1,25	2	O(n)
Відкрита адресація, ділення (m = 13)	0,62	1,25	1,80	2	O(n)
Відкрита адресація, множення (m = 16)	0,50	1,50	1,50	3	O(n)

Теоретичні оцінки середньої кількості порівнянь при успішному пошуку: для ланцюжків $1 + \alpha/2$, для лінійного дослідження $\frac{1}{2}\cdot(1 + 1/(1 - \alpha))$. З таблиці 4.7 видно, що в усіх чотирьох таблицях більшість слів знаходиться з першого порівняння, а середня кількість порівнянь становить від 1,25 до 1,50 при $n = 8$ ключах. Максимум у ланцюжках дорівнює 2 (слова «що», «коморі» для методу ділення та «полі», «а» для методу множення), а при відкритій адресації для методу множення слово «полі» потребує 3 порівнянь через кластер. Середня кількість порівнянь залежить від коефіцієнта заповнення α, а не від кількості елементів n, тому середня складність пошуку складає O(1). У найгіршому випадку, коли всі ключі мають однакове хеш-значення, пошук вироджується в перегляд усіх n елементів, тобто O(n). Експериментальні значення близькі до теоретичних; відхилення пояснюється малою кількістю ключів ($n = 8$), коли розподіл визначається кількома конкретними колізіями.

Висновки: Під час виконання четвертої лабораторної роботи я набув практичних навичок із проєктування, реалізації, тестування та аналізу

алгоритмів створення хеш-таблиць із роздільними ланцюжками та з відкритою адресацією (лінійне дослідження). Для 8 унікальних слів прислів'я я побудував таблиці з хеш-функціями методів ділення (m = 13) та множення (m = 16, A = 0,6180339887) і переконався, що результати Python -коду збігаються з ручними розрахунками. Обидві хеш-функції дали по дві колізії, які у ланцюжках розв'язувались додаванням у кінець списку, а при відкритій адресації – зсувом на наступну вільну комірку (з переходом через кінець таблиці для слова «коморі»). Середня кількість порівнянь при пошуку (видаленні) склала 1,25 і 1,25 для ланцюжків та 1,25 і 1,50 для відкритої адресації, максимум – 2 порівняння, а для відкритої адресації з методом множення – 3 (слово «полі»). Це підтверджує середню складність O(1) за невисокого коефіцієнта заповнення та складність O(n) у найгіршому випадку. Хешування з ланцюжками менш чутливе до кластеризації, а відкрита адресація не потребує покажчиків і економить пам'ять, але її ефективність швидше погіршується зі зростанням α.

Посилання на GitHub: https://github.com/9998920/algorithms-labs